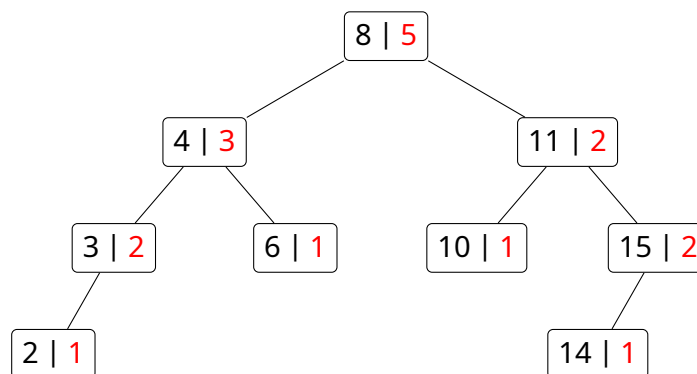


Encontrar el k -ésimo elemento en un árbol AVL

El objetivo de este ejercicio es extender el TAD de los conjuntos con una nueva operación k -ésimo que, dado un entero k , devuelva (si existe) el k -ésimo menor elemento del conjunto, es decir, el elemento que ocuparía la posición k si los elementos del conjunto se ordenaran. Por ejemplo, si k es 1 devolverá el menor elemento del conjunto; si k es 2, el segundo menor; y si k es igual al número de elementos del conjunto, devolverá su mayor elemento.

Con la implementación actual solo podríamos conseguir esa operación con un coste lineal en el número de elementos del conjunto, en el caso peor. Para poder hacerla más eficiente vamos antes a modificar la representación de los conjuntos mediante árboles AVL, añadiendo a cada nodo del árbol un nuevo atributo, tam_i , que almacene **el número de nodos del hijo izquierdo más uno**. Ese valor dice qué posición ocuparía la raíz del subárbol si sus elementos se ordenaran. En la siguiente figura aparece, en cada nodo, el elemento en negro y el valor de tam_i en rojo:



El tam_i de la raíz es 5, y en efecto el quinto menor elemento del conjunto $\{2, 3, 4, 6, 8, 10, 11, 14, 15\}$ es el 8.

Hay que modificar la implementación de la clase Set (archivo `TreeSet_AVL.h`) para añadir ese atributo y hacer los cambios necesarios para que las funciones que modifican los árboles mantengan coherente su valor. Para simplificar, se puede suponer que no hay operación de eliminación.

A continuación hay que extender la clase Set con un nuevo método

```
T const& kesimo(int k) const
```

que devuelva el k -ésimo menor elemento del conjunto, si existe; si no existe, la operación lanzará una excepción. El coste de esta nueva operación debe ser **logarítmico** respecto al número de elementos del conjunto.

Entrada

La entrada está formada por diversos casos de prueba. Cada caso ocupa cuatro líneas.

En la primera aparece el número N de valores (entre 1 y 300.000), **no necesariamente distintos**, a insertar en el conjunto. A continuación, en una misma línea, aparecen esos

valores (números enteros entre 1 y 1.000.000) separados por espacios, en el orden en que deben ser insertados. En la siguiente línea aparece el número M de elementos que se van a consultar (entre 1 y N). Y en la última línea aparecen, separadas por espacios, las M posiciones (ordinales) de los elementos a consultar (números entre 1 y N).

La entrada termina cuando un conjunto no tiene elementos (N es 0).

Salida

Para cada caso de prueba se escribirán, en líneas separadas, los elementos consultados, si existen. Si no hay elemento en la posición consultada, se escribirá ?? en su lugar.

Tras procesar cada caso se escribirá una línea más con tres guiones, - - -.

Entrada de ejemplo

```
4
15 20 25 30
2
1 3
5
16 8 4 4 32
3
2 4 5
1
7
1
1
0
```

Salida de ejemplo

```
15
25
- - -
8
32
??
- - -
7
- - -
```

Notas

Como los valores que se insertan no son necesariamente distintos, el conjunto puede tener menos elementos que valores se hayan leído, y entonces hay posiciones entre 1 y N que no corresponden a ningún elemento. Es lo que ocurre en el segundo caso del ejemplo: de los cinco valores leídos, el 4 aparece dos veces, así que el conjunto es {4, 8, 16, 32} y la quinta posición no existe.